

Tutorial 2: VAR Workflow & AI Literacy

Macroeconometrics, M1 APE

Juan Pablo Ugarte
Paris School of Economics

April 2026

Outline

1 Overview

2 VAR exercise

3 Structural analysis

4 AI literacy

5 Wrap-up

Where we are

- Tutorial 1: **reproducible environment**, data pipeline, AR(2) on US GDP growth
- Today: the **full VAR workflow** on quarterly US data + **AI literacy**
- After today: office hours (3 h/group) and the final project presentations

Today's objective

Be able to run a complete VAR: from FRED download to IRFs and Granger causality. A view on how to use AI tools **safely and productively** in your projects.

Today's roadmap

Part I – VAR exercise

1. VAR recap: why, what, how
2. Data pipeline: quarterly FRED series, transformations, stationarity
3. Lag selection and estimation
4. Stability and residual diagnostics
5. Structural interpretation: IRFs and FEVD

Part II – Extension: Granger causality

6. Granger test, interpretation, pitfalls

Part III – AI literacy

7. Ground rules + the 3-layer view (replicate, extend, defend)
8. Tool landscape and prompting patterns
9. Verification habits and academic honesty

From AR to VAR

An AR(p) for a single series y_t :

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t.$$

A **VAR**(p) generalises this to a *vector* $y_t \in \mathbb{R}^K$:

$$y_t = c + A_1 y_{t-1} + \cdots + A_p y_{t-p} + \varepsilon_t, \quad \varepsilon_t \sim WN(0, \Sigma).$$

Why we care

- Joint dynamics: feedback effects between variables of interest
- Policy questions: “what happens in the economy if the Fed tightens?”
- A flexible benchmark for macro forecasting

What we will build today

- $K = 3$: inflation, unemployment rate, Fed funds rate
- Quarterly, 1960Q1–2007Q4 (pre-GFC)
- Stock & Watson (2001) specification
- Cholesky IRFs, FEVD, Granger tests

Workflow: five steps

1. **Data**: download, transform, check stationarity
2. **Lag**: select p via information criteria
3. **Estimate**: OLS equation-by-equation (VAR() from vars)
4. **Check**: stability, serial correlation, normality
5. **Interpret**: IRFs, FEVD, Granger causality

Keep this in mind

The VAR is **reduced-form**: innovations ε_t are correlated across equations. Any *structural* interpretation requires an identifying assumption (today: Cholesky).

Outline

- 1 Overview
- 2 VAR exercise
- 3 Structural analysis
- 4 AI literacy
- 5 Wrap-up

Step 1: Download quarterly FRED data

Three classic variables for a monetary VAR, following **Stock & Watson (2001, JEP)**:

- GDPDEF – GDP implicit price deflator (prices)
- UNRATE – Civilian unemployment rate (real activity, slack)
- FEDFUNDS – Effective Fed funds rate (monetary policy)

```
library(fredr); library(dplyr); library(tidyr)
fredr_set_key(Sys.getenv("FRED_API_KEY"))

series_ids <- c("GDPDEF", "UNRATE", "FEDFUNDS")

raw_list <- lapply(series_ids, function(id) {
  fredr(series_id = id,
        observation_start = as.Date("1955-01-01"),
        observation_end = as.Date("2007-12-31"),
        frequency = "q")      # quarterly aggregation
})

macro <- raw_list |>
  lapply(function(df) df[, c("date", "value")]) |>
  Reduce(function(x, y) merge(x, y, by = "date"), x = _) |>
  setNames(c("date", "pgdp", "unrate", "ffr"))
```


Step 1 (cont.): Transform and build the VAR dataset

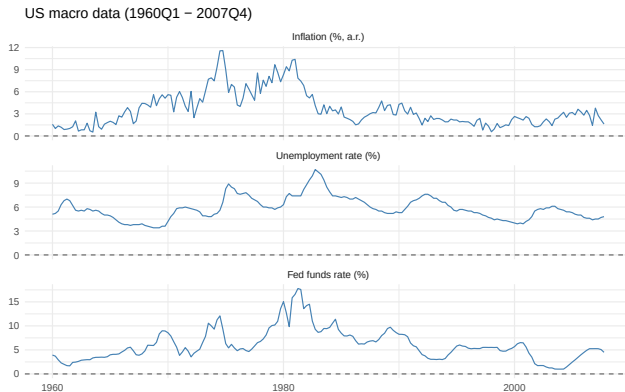
```
# Inflation: annualised log-difference of the GDP deflator.
# Unemployment and Fed funds rate enter in levels.
macro <- macro |>
  mutate(infl = c(NA, 400 * diff(log(pgdp)))) |> # % annualised
  tidyr::drop_na() |>
  filter(date >= as.Date("1960-01-01"),
         date <= as.Date("2007-12-31"))          # 192 quarters

# Cholesky ordering (Stock & Watson 2001):
#   infl (slowest), unrte, ffr (fastest)
var_data <- macro |> select(infl, unrte, ffr)
var_ts   <- ts(var_data,
               start = c(year(min(macro$date)), quarter(min(macro$date))),
               frequency = 4)
```

Why this ordering?

Cholesky assumes a **recursive** structure. A shock to ffr does not move infl or unrte *within the quarter*, but the Fed *does* react contemporaneously to inflation and unemployment news.

Step 1 (cont.): Visualize the series



- The 1970s – Great Inflation; Volcker disinflation around 1980–82.
- Unemployment moves counter-cyclically around $\sim 6\%$ with clear recession spikes.
- All three series are highly **persistent** – expect large AR roots.

Step 2: Stationarity tests (PP, DF-GLS, KPSS)

```
library(tseries)      # pp.test, kpss.test
library(urca)         # ur.ers (DF-GLS)

for (j in seq_along(colnames(var_ts))) {
  y <- var_ts[, j]
  pp <- pp.test(y, alternative = "stationary")
  ers <- ur.ers(y, type = "DF-GLS",
               model = "constant")
  kp <- kpss.test(y, null = "Level")
  # store stat + p-value (or 5% CV for DF-GLS)
}
print(ur_tbl)
```

Output:

	PP		DF-GLS		KPSS	
	stat	p	stat	cv5	stat	p
infl	-15.7	0.22	-1.78	-1.94	0.86	0.010
unrate	-11.4	0.47	-2.31	-1.94	0.47	0.048
ffr	-11.8	0.44	-2.32	-1.94	0.64	0.019

Three complementary tests

- PP: $H_0 =$ unit root.
- DF-GLS: $H_0 =$ unit root.
- KPSS: $H_0 =$ **stationary**.

Reading the table

- PP fails to reject for all three
- DF-GLS rejects for unrate, ffr (below -1.94); not for infl
- KPSS rejects stationarity everywhere

Evidence of high persistence, but **tests disagree** for unrate/ffr. We proceed with the levels (Stock-Watson practice) and flag this in the diary.

Step 3: Lag order selection

```
library(vars)

lag_sel ← VARselect(var_ts,
                    lag.max = 8,
                    type     = "const")

lag_sel$selection
```

Output:

AIC(n)	HQ(n)	SC(n)	FPE(n)
8	3	2	6

Criteria **disagree**. We follow HQ and set $p^* = 3$ a middle ground between AIC and BIC.

Criteria in VARselect

- AIC: Akaike
- HQ: Hannan-Quinn
- SC: Schwarz (BIC)
- FPE: Final Prediction Error

When they disagree

- BIC is consistent but often too sparse for macro VARs
- AIC/FPE tend to pick larger p (better short-horizon fit)
- HQ is also consistent but less aggressive than BIC

Step 4: Estimate the VAR

```
p_star <- 3
var_fit <- VAR(var_ts, p = p_star, type = "const")
summary(var_fit)
```

Output (FFR equation):

	Estimate	Std.Err	t	value	Pr(> t)
infl.l1	0.0632	0.0641	0.99	0.3254	
unrate.l1	-1.3354	0.3002	-4.45	1.5e-05 ***	
ffr.l1	0.9817	0.0799	12.29	<2e-16 ***	
infl.l2	0.2104	0.0738	2.85	0.0049 **	
unrate.l2	0.9083	0.4840	1.88	0.0622 .	
ffr.l2	-0.4157	0.1100	-3.78	0.0002 ***	
infl.l3	-0.1016	0.0660	-1.54	0.1254	
unrate.l3	0.3130	0.2823	1.11	0.2690	
ffr.l3	0.3612	0.0809	4.47	1.4e-05 ***	
const	0.4918	0.2763	1.78	0.0768 .	

Reading the FFR equation

- FFR is very persistent ($\sum \phi_{\text{ffr}} \approx 0.93$)
- Unemployment at lag 1 enters *negatively* $\Rightarrow$ the Fed tightens when unemployment is low (Taylor rule)
- Inflation has a positive lagged effect at lag 2 – also Taylor-rule-like
- Each of the $K = 3$ equations is estimated by OLS

Step 5: Stability

A VAR(p) is **stable** iff all eigenvalues of the companion matrix lie inside the unit circle.

```
roots_mod ← roots(var_fit, modulus = TRUE)
max(roots_mod)
# [1] 0.961
```

- Largest root has modulus $0.96 < 1 \Rightarrow$ the VAR is stable
- But 0.96 is close to 1 – consistent with very persistent data
- Stability is *necessary* for IRFs to die out and for standard inference to apply

If the VAR is not stable...

You likely have a unit root left in at least one variable. Go back to step 1: transform more, or difference again.

Step 5 (cont.): Residual diagnostics

```
serial.test    (var_fit, lags.pt = 12, type = "PT.asymptotic") # Portmanteau  
normality.test(var_fit, multivariate.only = TRUE)             # Jarque-Bera
```

Output (selected):

```
Portmanteau Test: Chi-squared = 150.63, df = 81, p-value = 4.26e-06  
JB multivariate:  Chi-squared = 565.90, df =  6, p-value < 2.2e-16
```

Reading the tests

- Serial corr.: strongly rejected, residuals are not white noise.
- Normality: rejected (fat tails, typical for macro data).

What to do

- Larger p (e.g., 8) often removes serial corr. (at the cost of parameters).
- Use **bootstrap** inference for IRFs (robust to non-normality).
- Or restrict the sample to the Great Moderation (1985Q1–).
- **Document** the trade-off in your diary.

Outline

- 1 Overview
- 2 VAR exercise
- 3 Structural analysis**
- 4 AI literacy
- 5 Wrap-up

Step 6a: Impulse Response Functions

```
irf_ffr <- irf(var_fit,
  impulse = "ffr",
  response = c("infl", "unrate", "ffr"),
  n.ahead = 20,
  ortho = TRUE,      # Cholesky orthogonalisation
  boot = TRUE,
  runs = 500,
  ci = 0.90)
```

What this computes:

- A +1 s.d. orthogonal shock to ffr (monetary tightening)
- Dynamic response of every variable for 20 quarters
- 90% bands from bootstrap (500 draws)

Identification assumption

Because of Cholesky, shocks to ffr are orthogonal to (and ordered after) shocks to inflation and unemployment. This embeds the assumption that the Fed reacts **within the quarter**, but inflation and unemployment do **not** contemporaneously react to the policy shock.

Step 6a: IRFs – output

Responses to a monetary policy shock (+1 s.d. to FFR)

Cholesky IRFs, 90% bootstrap bands



- **Unemployment** rises with a hump peaking at ~ 8 quarters, a classic real effect of MP
- **Inflation** shows a small short-run “price puzzle” then drifts below zero
- FFR shock dies out smoothly over ~ 5 years, consistent with the large AR roots

Step 6b: Forecast Error Variance Decomposition

```
fevd_fit <- fevd(var_fit, n.ahead = 20)  # Shares per shock, per variable
```



- Inflation: almost entirely driven by its own shocks ($\sim 88\%$ at $h = 20$) – strong persistence.
- Unemployment: FFR shocks explain $\sim 13\%$ of forecast errors at $h = 20$.
- FFR: at long horizons, inflation and unemployment shocks jointly dominate – the Fed *endogenously* responds to both.

Granger causality: what it is (and isn't)

Definition. x *Granger-causes* y if past values of x help predict y *beyond* what is already in past values of y (and of the other variables in the system).

What it is

- A test of **predictive content**
- Run in the *reduced-form* VAR
- Simple F -test on a block of lag coefficients

What it is NOT

- Not causality in the structural sense
- Not immune to omitted variable bias
- Absence of Granger causality $\neq$ irrelevance

Useful for your project

A natural **extension** to a replication: “Does variable X Granger-cause the outcome of interest, controlling for the original set?”

Granger causality in R

```
# Block test: does "ffr" Granger-cause the other variables?  
gc_ffr <- causality(var_fit, cause = "ffr")  
gc_ffr$Granger
```

Output:

```
Granger causality H0: ffr do not Granger-cause infl unrate  
  
data:  VAR object var_fit  
F-Test = 3.21, df1 = 6, df2 = 537, p-value = 0.00416
```

Full results (our three block tests):

Cause (block)	F-stat	p-value
ffr → {inflation, unemployment}	3.21	0.00416
unrate → {inflation, FFR}	6.16	2.9×10^{-6}
infl → {unemployment, FFR}	5.86	6.2×10^{-6}

Granger causality: reading the results

- **FFR** $\rightarrow$ **{inflation, unemployment}**: significant at 1%. Consistent with the IRF: monetary policy has predictive content for the other two variables
- **Unemployment** $\rightarrow$ **{inflation, FFR}**: highly significant. The Fed systematically responds to labour-market slack, and unemployment forecasts inflation (Phillips-curve logic)
- **Inflation** $\rightarrow$ **{unemployment, FFR}**: highly significant. Past inflation predicts both real activity and the policy rate – the Taylor-rule channel

What you should write in your diary

- Why you picked that cause block (economic question)
- What H_0 you tested and under what assumptions
- What changes if you re-order, change sample, or change lag

Outline

- 1 Overview
- 2 VAR exercise
- 3 Structural analysis
- 4 AI literacy**
- 5 Wrap-up

AI in your VAR project: the ground rules

AI tools are allowed and encouraged. But the work has to be *yours*.

What AI is good at

- Organising scripts and setup
- Data wrangling idioms (reshape, joins)
- Debugging R / $\text{\LaTeX}$ error messages
- Explaining unfamiliar package syntax
- Generating plots with known specs (IRFs, FEVD)

What AI is bad at

- Knowing the *exact* FRED code / indicator
- Matching the specification precisely (lag, ordering, sample)
- Deciding *what question* to ask
- Interpreting *your* IRFs in economic terms
- Defending judgement calls in Q&A

Rule of thumb

If you can't explain a line of code or a modeling choice in your own words, **it shouldn't be in your project.**

Replicate, extend, defend

Layer 1: Replication – AI can genuinely accelerate you.

- Organising scripts, setting up the environment, debugging vars/urca errors.
- **Always verify:** FRED codes, sample period, units, package argument names.

Layer 2: Extension – AI helps, but you drive.

- AI will happily generate a plausible-looking extension that is *not interesting*.
- *You* decide what to vary (sample, identification, lag order) and why.

Layer 3: Defense – AI is useless.

- The oral presentation is where this becomes visible.
- If you didn't write it, you won't be able to defend it.

Common failures in a VAR context

Hallucinated vars/urca arguments, wrong FRED codes, confident-sounding but wrong IRF interpretation, fabricated citations, uninteresting extensions.

The AI coding tool landscape

Roughly three kinds of tools. Use whichever you're comfortable with.

1. **Chat interfaces** – ChatGPT, Claude.ai, Gemini, Mistral

- Copy-paste, read the answer. No access to your files.
- *Best for:* explanations, one-off snippets, decoding an error.

2. **In-editor completions** – GitHub Copilot, Cursor Tab, Codeium, Supermaven

- Inline suggestions as you type, inside VSCode / RStudio / etc.
- *Best for:* syntax, mechanical typing.

3. **Repo-aware / agent-mode** – Cursor, Claude Code, Windsurf, Aider

- Read and edit multiple files; run commands and iterate on output.
- *Best for:* scaffolding, multi-file refactors, debugging. Close review required.

Any of these is fine. Free tiers (ChatGPT, Claude.ai, Gemini) or GitHub Copilot (free for students via GitHub Education) are plenty. Don't pay for a subscription just for this project!

Prompts that work, prompts that don't

Whatever tool you use, the prompt determines quality. Be specific, give context, point at files.

Prompts that work well:

- *"Set up renv, install the packages loaded at the top of code.R, and snapshot."*
- *"Read code.R and explain the irf() block step by step."*
- *"I get this error when I run VAR(): What's going on?"*
- *"Add an AIC-selected-lag column to my VARselect summary table."*

Prompts that don't:

- *"Replicate Stock & Watson 2001."* (too vague; invents codes and specifications)
- *"Is my extension interesting?"* (AI will always say yes)
- *"What's the right identification scheme?"* (judgment call, you commit to one)
- *"Fix my code."* (without saying what failed and what you tried)

AI practices for your project

Do

- Read the diff before accepting AI edits.
- Check function signatures: ?VAR, ?irf.
- Plot every object; verify numbers are on the expected scale.
- Cross-check FRED codes and package arguments against the source.
- Store exchanges in the modeling diary:
*“Asked AI to refactor the IRF plot;
accepted because . . .”*

Don't

- Paste data into a chat and trust the numbers it reports back.
- Accept multi-file edits without carefully reading them.
- Ask for “a good extension” and run with the first suggestion.
- Let AI interpret your IRFs before you've looked at them.
- “Fix” failing diagnostics until the test passes – that's *p*-hacking.

Bottom line

AI is a **productivity tool for a student who already knows what they're doing**. Disclose AI use in your report, and defend every decision yourself.

Outline

- 1 Overview
- 2 VAR exercise
- 3 Structural analysis
- 4 AI literacy
- 5 **Wrap-up**

Today's checklist

By the end of today, you should be able to:

- Download quarterly FRED data and transform it to stationary form
- Select a VAR lag order and estimate the model
- Run all standard diagnostics (stability, serial correlation, normality)
- Compute and plot Cholesky IRFs and FEVD, and explain the identification
- Run and interpret a Granger causality test
- Use AI tools productively and verify their output

Before we finish

1. Finalize your paper choice (if not done already)
2. Run today's VAR model on your computer
3. Schedule your office-hours and presentation slots: <https://tinyurl.com/5xj5exye>
4. Try at least one AI coding tool on a piece of your replication. Notice where it helps and where it doesn't.

Key dates:

- Presentations: **May 26 and 27**
- Final report: **May 29**

Questions?